CSE 211 — Theory of Computation

Final study guide: the method for every question type, with past-paper questions answered as you would write them

Part A: Hakim Sir (Topics 1–6) · Part B: Atif Sir (Topics 7–12) · Term final: 10 November 2026

How this guide is built. Every topic opens with *what they ask* (counted from the practice-tracker papers, 2017-18 to 2023-24), then a blue *recipe*, then green *exam answers*: a real past question in the grey upper half and, below the line, the answer written the way it should appear in your script — design idea, construction, table or diagram, a check, and a closing sentence. Red boxes are the mistakes that lose marks.

Companions. Full proofs: `CSE211_Proofs_Exam_Ready.pdf`. Every tracker question with a solution: `CSE211_Hakim/Atif_TF_Questions_Solutions.pdf`.

Checked by program (`_build/verify_guide_v2.py`, `_build/verify_final_guide.py`): every DFA, NFA, regular expression, grammar, CYK table, PDA and Turing machine in the exam answers, on all strings up to length 7–12.

How to write any answer for full marks

1. **Idea first, in one or two sentences** ("the state remembers …", "the stack holds one A per a …"). Several questions say rules alone get no marks.
2. **Name every state / variable / stack symbol and say what it means.** Mark start ($\to$) and accepting ($*$ or double circle) states.
3. **Give the full object:** diagram *and* table when asked; every DFA state has a move on every symbol (dead state).
4. **Check on the string the question names and on one string that must fail**; show the trace (states, IDs, CYK table, tape).
5. **End with one sentence that answers the question** ("Hence $0100 \in L(G)$", "So L is not regular").
6. **Time:** roughly 1.5 minutes per mark; a 15-mark design question is about 20 minutes. Do the design and the trace before polishing.

	Topic	Tracker Qs	The question that keeps coming back
Hakim	1 Introduction	4	"which area of ToC / why hard problems are useful" (Q1(a), 4–15 marks)
	2 DFA	13	design a DFA (counter, mod-m, forbidden substring); describe a DFA's language
	3 NFA	13	design an NFA; ε-NFA $\to$ DFA; strings of length ≤ 3 accepted
	4 Regular languages	24	closure constructions with δ; choose/design/simplify regular expressions; RE $\to$ NFA
	5 Pumping lemma	9	prove non-regular; "what is wrong with Skye's/BLP's argument"
	6 CFG	15	design a CFG (with design idea); show a grammar is ambiguous
Atif	7 CNF and CYK	7	CYK on a CNF grammar (5 papers); CNF conversion (2)
	8 PDA, CFL pumping	10	design a PDA with diagram (every paper); final state = empty stack; non-CFL proof
	9 TM design	6	TM on tape w_1cw_2c: AND / OR / XOR / equality / NOT / 2's complement (every paper)
	10 TM variants	6	one-tape TM simulates multitape; true/false on simulations (every paper)
	11 Undecidability	7	L_d not RE, or halting problem RE but not recursive (every paper)
	12 Complexity	17	Cook's theorem clause (4 papers), Venn diagram / PSPACE = NPSPACE (every paper), NP-completeness

Contents

Part A — Hakim Sir 3

1 Introduction: what Theory of Computation is for 3

2 DFA design 3

3 NFA, ε-moves and subset construction 5

4 Regular languages: closure, regular expressions, state elimination **6**

5 Pumping lemma (regular) **7**

6 Context-free grammars: design and ambiguity **8**

Part B — Atif Sir **10**

7 CNF and the CYK algorithm **10**

8 Pushdown automata and the CFL pumping lemma **10**

9 Turing machine design on $w_1 c w_2 c$ **12**

10 Turing-machine variants and simulations **13**

11 Undecidability **14**

12 Complexity: Cook's theorem, NP-completeness, space **15**

Part A — Hakim Sir

1 Introduction: what Theory of Computation is for

What they ask: 2023-24 Q1(a), 2022-23 Q1(a), 2021-22 Q1(a), 2019-20 Q1(b): short essay answers, 4–15 marks.

Recipe — "which area / why hard problems" questions

1. Name the three central areas: **automata** (what can be done with limited memory), **computability** (what can be solved at all), **complexity** (what can be solved efficiently).
2. Say what the system must *do* and match it: bounded-memory pattern checking → automata (DFA); "does any algorithm exist" → computability; "how much time / is it infeasible" → complexity.
3. Give the concrete model or class, and the consequence (hardness is *useful* for security).

Exam answer — 2021-22 Q1(a) (4 marks)

In the password-protected login system of a computer, which of the three traditionally central areas of the Theory of Computation has been applied? Explain clearly.

Answer. Two areas are applied.
Automata theory — checking that a new password obeys the rules (e.g. at least 8 characters, a digit, an upper-case letter) needs only a bounded memory of the prefix read: $\min(\text{length}, 8)$, "digit seen", "upper case seen". So the set of valid passwords is a regular language and is checked by a DFA in one left-to-right pass.
Complexity theory — the system stores $h(\text{password})$ for a one-way hash h. Computing h is easy (polynomial time), but finding a password with a given hash is believed infeasible. Security rests on this gap between easy and hard.
Computability plays no role: every task here is decidable.

Exam answer — 2022-23 Q1(a) and 2019-20 Q1(b) (10 / 15 marks)

Explain public and private keys in asymmetric cryptography and the role of ToC; describe a scenario that specifically needs computationally hard problems.

Answer. In asymmetric cryptography each user has a *public key* (published, used to encrypt or to verify signatures) and a *private key* (kept secret, used to decrypt or to sign). In RSA the public key contains $n = pq$ and the private key needs p, q.
Role of ToC (complexity). Encryption with the public key is easy (polynomial time). Decrypting without the private key amounts to factoring n, for which no polynomial-time algorithm is known. A proposed key is easy to *check* (multiply $p \cdot q$) but hard to *find* — the same "verify easy / find hard" gap as NP versus P. If P = NP, such schemes would collapse.
Scenario that needs hard problems: online banking / HTTPS key exchange, digital signatures, password hashing, CAPTCHAs and proof-of-work. Here hardness is a *resource*: the system is safe exactly because the attacker's problem is hard.

Exam answer — 2023-24 Q1(a) (8 marks)

What options do we have when a problem appears to be computationally hard?

Answer. (1) *Prove it hard* (NP-complete) so we stop looking for an exact fast algorithm. (2) *Approximation*: a polynomial algorithm with a guaranteed ratio (e.g. vertex cover within factor 2). (3) *Heuristics / local search*: good answers fast, no guarantee. (4) *Special cases*: restrict the input (trees, bounded degree, 2-SAT is in P). (5) *Exact exponential algorithms* for small inputs (backtracking, branch and bound, dynamic programming). (6) *Randomisation* and average-case analysis. (7) Use the hardness: cryptography.

2 DFA design

What they ask: 13 questions. Design (2023-24 Q1(b), Q2(b); 2022-23 Q1(c), Q3(a); 2021-22 Q1(b); 2019-20 Q1(c),

Q2(c); 2017-18 Q2(b)); describe the language of a given DFA (2022-23 Q2(a), 2020 Q2(a)); theory (role of a state, accepting states and the table).

Recipe — designing a DFA

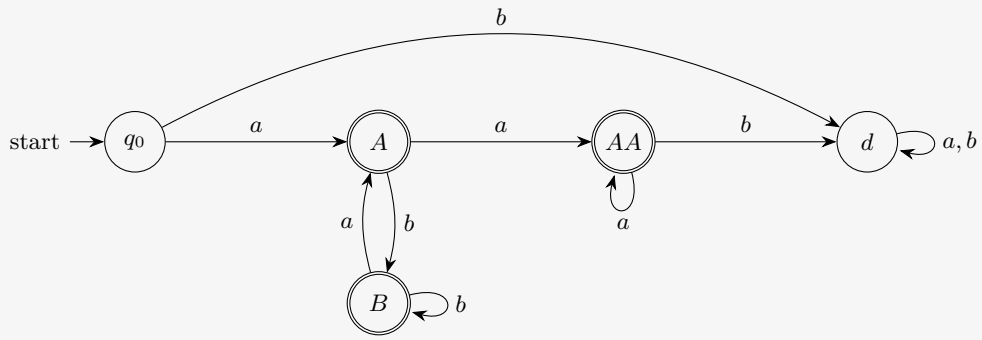
1. Ask *what is the least I must remember about the prefix read so far?* Each possible value is a state. Typical memories: a count capped at k, a remainder mod m, the last one or two symbols, a phase, "already failed" (dead state).
2. Start state = memory of ε. Accepting states = memory values meaning "accept".
3. Fill δ for *every* (state, symbol) pair; send impossible continuations to one dead state.
4. Draw the diagram, write the table if asked, test the named strings, ε, and one rejected string.

Numbers in base b mod m: digit d turns value v into $bv + d$, so $\delta(r, d) = (br + d) \bmod m$. **Running sum with reset R:** $\delta(r, d) = (r + d) \bmod m$, $\delta(r, R) = 0$.

Exam answer — 2021-22 Q1(b) (10 marks)

Design a DFA with not more than five states for $\Sigma = \{a, b\}$ which accepts the strings that begin with a but do not contain the substring aab. Showing only the state diagram should suffice.

Design idea. The first symbol must be a (a first b goes to a dead state). After that, aab appears exactly when a b is read while the input ends in aa, so it is enough to remember whether the input ends in b, in a single a, or in aa.
States. q_0: nothing read. A: ends in a single a. AA: ends in aa. B: ends in b (the start was fine). d: dead (began with b, or aab seen). Accepting: A, AA, B. Five states.



Check. $abab$: $q_0 \to A \to B \to A \to B$ accept. $aaab$: $q_0 \to A \to AA \to AA \xrightarrow{b} d$ reject (contains aab). ba: $q_0 \xrightarrow{b} d$ reject. aaa: ends in AA, accept. Hence the DFA accepts exactly the required language.

Exam answer — 2019-20 Q1(c) (15 marks)

Draw a DFA over $\{0, 1, 2, R\}$ that keeps the running sum of the digits, resetting it to 0 on R; accept if the sum is 1 or 2 modulo 5.

Design idea. Only the sum mod 5 matters, so five states $s_0, \ldots, s_4$ (s_i: sum $\equiv i$). Digit d: $s_i \to s_{(i+d) \bmod 5}$; R: every state $\to s_0$. Start s_0; accepting s_1, s_2.

	0	1	2	R
$\to s_0$	s_0	s_1	s_2	s_0
$*s_1$	s_1	s_2	s_3	s_0
$*s_2$	s_2	s_3	s_4	s_0
s_3	s_3	s_4	s_0	s_0
s_4	s_4	s_0	s_1	s_0

(Draw the five states in a ring: each 0 is a self-loop, 1 goes one step round, 2 goes two steps round, every R returns to s_0.) **Check.** $12R2$: $s_0 \to s_1 \to s_3 \xrightarrow{R} s_0 \to s_2$ accept (sum after reset is 2). 22: $s_0 \to s_2 \to s_4$ reject (sum 4).

Trap. Every state needs an arrow for every symbol — draw the dead state. Accept ε only if the language contains it. For "binary multiple of m beginning with 1" add a separate start state (a leading 0 goes to dead). **Theory answers:** a state is a *summary of everything relevant about the input read so far* (finite memory); accepting states do *not* change the rows or columns of the transition table (rows = states, columns = symbols), they are only marked with $*$. **2022-23 Q1(b):** a 2-state DFA ("remainder 0 / non-zero") cannot track a sum mod 3: after "1" and after "2" it would be in the same state, yet appending "2" gives sums 3 (remainder 0) and 4 (remainder 1). Strings that must be told apart need different states.

3 NFA, ε-moves and subset construction

What they ask: 13 questions. Design an NFA (2023-24 Q3(a), Q4(a); 2022-23 Q4(b); 2021-22 Q3(b); 2019-20 Q3(b); 2017-18 Q3(a)); convert an ε-NFA to a DFA (2021-22 Q3(c), 2020 Q4(b), 2017-18 Q4(a)); list accepted strings of length ≤ 3 (2020 Q1(c)).

Recipe — NFA design and NFA → DFA

Design: let the NFA *guess* the moment something happens ("the suffix starts here"); for "X or Y" build two small machines and join them with ε-edges from a new start state.

ε-NFA → DFA (subset construction):

1. Write $E(q)$, the ε-closure of every state. DFA start state = $E(q_0)$.
2. For each new subset S and symbol a: $\delta_D(S,a) = E\big(\bigcup_{q\in S}\delta(q,a)\big)$. Keep going until no new subset appears.
3. Only *reachable* subsets; $\emptyset$ is the dead state. Accepting = subsets containing an accepting NFA state.
4. Show the table, then draw the DFA.

Exam answer — 2021-22 Q3(c) **(9 marks)**

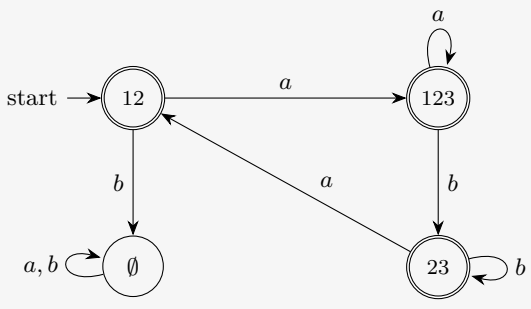
Convert the following NFA into an equivalent DFA (start q_1, accepting q_2): q_1: $a \to \{q_3\}$, $\varepsilon \to \{q_2\}$; q_2: $a \to \{q_1\}$; q_3: $a \to \{q_2\}$, $b \to \{q_2, q_3\}$.

ε-closures. $E(q_1) = \{q_1, q_2\}$, $E(q_2) = \{q_2\}$, $E(q_3) = \{q_3\}$. DFA start state = $E(q_1) = \{q_1, q_2\}$.

Subset table.

DFA state	on a	on b
$\to *\{q_1, q_2\}$	$E(\{q_3\} \cup \{q_1\}) = \{q_1, q_2, q_3\}$	$\emptyset$
$*\{q_1, q_2, q_3\}$	$E(\{q_3, q_1, q_2\}) = \{q_1, q_2, q_3\}$	$E(\{q_2, q_3\}) = \{q_2, q_3\}$
$*\{q_2, q_3\}$	$E(\{q_1, q_2\}) = \{q_1, q_2\}$	$E(\{q_2, q_3\}) = \{q_2, q_3\}$
$\emptyset$ (dead)	$\emptyset$	$\emptyset$

Accepting: every subset containing q_2, i.e. the three non-empty ones. Four reachable states out of $2^3 = 8$.

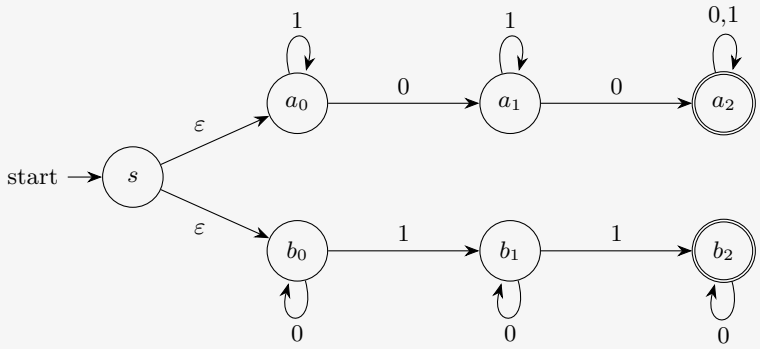


Check. ε: accepted in the NFA ($q_1 \xrightarrow{\varepsilon} q_2$) and in the DFA (start is accepting). b: the NFA has no b-move from q_1, q_2, the DFA goes to $\emptyset$: both reject. So the DFA is equivalent to the NFA.

Exam answer — 2021-22 Q3(b) **(10 marks)**

Design an NFA which accepts binary strings containing at least two 0s, or exactly two 1s. Showing only the state diagram should suffice.

Design idea. Union of two counters: a new start state s guesses with ε-edges which condition will hold. Upper branch counts 0s up to 2 (1s are ignored); lower branch counts 1s and has *no* move on a third 1, so that branch dies.



Check. 0110: upper branch reaches a_2 (two 0s) — accept. 1011: one 0, three 1s; upper ends in a_1, lower dies on the third 1 — reject. 101: lower ends in b_2 — accept.

Trap. Never list all 2^n subsets — only reachable ones. Take the ε-closure at the start *and* after every move. "Accepted strings of length ≤ 3" (2020 Q1(c)): follow every branch for each length 0, 1, 2, 3 and write the general form (e.g. "any string ending in c"). A 5-state NFA gives a DFA with between 1 and $2^5 = 32$ states.

4 Regular languages: closure, regular expressions, state elimination

What they ask: 24 questions — the biggest topic. Closure constructions with δ and reasons (2019-20 Q3(c), 2021-22 Q4(b), 2020 Q3(b), 2023-24 Q3(b), Q1(c)); complement by swapping (2017-18 Q2(a), 2021-22 Q1(c), 2022-23 Q3(b)); choose / design / simplify regular expressions (2023-24 Q4(b), 2022-23 Q4(c), 2021-22 Q3(d), 2019-20 Q4(b), 2017-18 Q4(b), 2020 Q2(c), Q3(a), Q4(a)); RE → NFA by basic principles (2021-22 Q2(b), 2019-20 Q3(a), 2017-18 Q2(c)).

Recipe — the five constructions to know by heart (proofs: Proofs PDF, Section 2–3)

Union: new start q_0, $\delta(q_0, \varepsilon) = \{q_1, q_2\}$, keep both machines, $F = F_1 \cup F_2$. **Concatenation:** ε from every $f \in F_1$ to q_2; $F = F_2$. **Star:** *new* accepting start $q_0 \xrightarrow{\varepsilon} q_1$, and ε from every $f \in F_1$ back to q_1. **Complement:** complete DFA, swap F and $Q \setminus F$ (not for NFAs). **DFA → RE:** add new start/accept, remove states using $R_{ij} \cup R_{i,rip} R^*_{rip,rip} R_{rip,j}$.
Designing an RE: describe the strings as blocks ("groups of three a's with b's anywhere"), write one block, then star or concatenate. Check one accepted and one rejected string, and the edge cases (ε, all-b).
Choosing among given REs: for each option give a string it wrongly accepts or a string of L it misses.

Exam answer — 2019-20 Q4(b) **(15 marks)**

Over $\{a, b\}$ write regular expressions for (i) strings whose number of a's is divisible by 3; (ii) strings with no more than three a's; (iii) strings with exactly one occurrence of the substring aaa.

(i) Group the a's in threes with b's anywhere: $b^*(ab^*ab^*ab^*)^*$. (ε, bbb, aaa, $abaab$ are in; aa is not.)
(ii) At most three optional a's separated by b's: $b^*(a \cup \varepsilon)b^*(a \cup \varepsilon)b^*(a \cup \varepsilon)b^*$.
(iii) "Exactly one occurrence" counts overlaps ($aaaa$ has two), so the string has exactly one maximal a-block of length exactly 3 and every other a-block has length ≤ 2. Strings with no aaa that are empty or end in b: $N = (b \cup ab \cup aab)^*$. Answer:

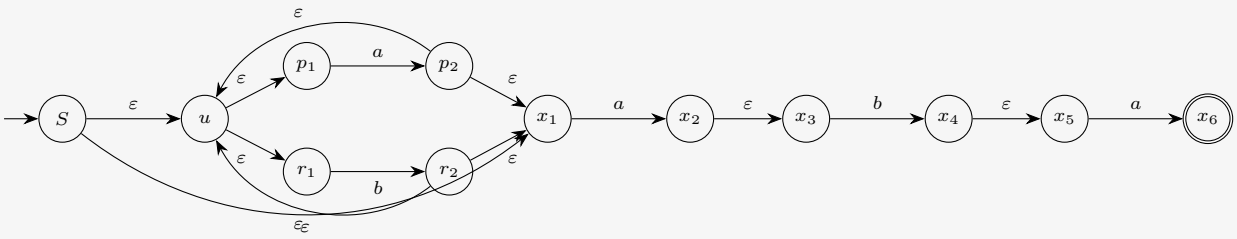
$$N\,aaa\,\big(\varepsilon \cup b\,N\,(\varepsilon \cup a \cup aa)\big), \qquad N = (b \cup ab \cup aab)^*.$$

Check: $baaab$ ($N = b$, then aaa, then b) is in; $aaaa$ is not (aaa must be followed by b or the end); $aaabaaa$ is not (the part after b is in $N(\varepsilon \cup a \cup aa)$, which has no aaa).

Exam answer — 2017-18 Q2(c) **(8 marks)**

Using the techniques of proving closure properties of regular languages, draw the NFA for the regular expression $(a \cup b)^*aba$.

Construction (bottom-up). (1) a: $p_1 \xrightarrow{a} p_2$; b: $r_1 \xrightarrow{b} r_2$. (2) $a \cup b$: new start u with $u \xrightarrow{\varepsilon} p_1$, $u \xrightarrow{\varepsilon} r_1$. (3) $(a \cup b)^*$: new *accepting* start S, $S \xrightarrow{\varepsilon} u$, and $p_2, r_2 \xrightarrow{\varepsilon} u$. (4) Concatenate with a, b, a ($x_1 \xrightarrow{a} x_2$, $x_3 \xrightarrow{b} x_4$, $x_5 \xrightarrow{a} x_6$): ε from the accepting states S, p_2, r_2 of the star part to x_1, then $x_2 \xrightarrow{\varepsilon} x_3$, $x_4 \xrightarrow{\varepsilon} x_5$. Only x_6 accepts.



Check. *aba*: $S \xrightarrow{\varepsilon} x_1 \xrightarrow{a} \xrightarrow{\varepsilon} \xrightarrow{b} \xrightarrow{\varepsilon} \xrightarrow{a} x_6$ accept; *baba*: one loop through $r_1 \to r_2$, then $\to x_1$ and *aba* — accept. *ab* never reaches x_6 — reject. (Simplified equivalent NFA: $s \xrightarrow{a,b} s$, $s \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{a} 3$, accept 3.)

Exam answer — State elimination (2022-23 Q4(c) type) **(10 marks)**

Find a regular expression for the DFA over $\{0, 1\}$ with states E (start, accepting) and O, where 0 loops on both states and 1 swaps them.

Step 1 (GNFA). New start $s \xrightarrow{\varepsilon} E$, new accept $E \xrightarrow{\varepsilon} f$. Labels: $E \to E : 0$, $E \to O : 1$, $O \to O : 0$, $O \to E : 1$.
Step 2 (rip O). Only path through O: $E \to O \to E$, label $1\,0^*\,1$. New loop on E: $R(E, E) = 0 \cup 10^*1$.
Step 3 (rip E). $R(s, f) = \varepsilon\,(0 \cup 10^*1)^*\,\varepsilon$.
Answer. $(0 \cup 10^*1)^*$ = strings with an even number of 1s. Check: 11, 0101 in; 1, 0111 out.

Short theory answers (2–12 marks each) — write these sentences

- $\emptyset^* = \{\varepsilon\}$ (2022-23 Q3(c), 2021-22 Q3(a)): A^* includes the concatenation of *zero* strings, ε, for any A; one or more pieces would need a string from $\emptyset$, impossible.
- **Swap accept states** (2017-18 Q2(a), 2021-22 Q1(c), 2022-23 Q3(b)): works for a *complete* DFA (one run per string, so it ends in F or not). Fails without a dead state, and fails for NFAs: $q_0 \xrightarrow{a} q_0$, $q_0 \xrightarrow{a} q_1$, $F = \{q_1\}$ accepts a^+; swapped it accepts a^*, not $\{\varepsilon\}$.
- **Star without a new start state** (2023-24 Q3(b)): N_1 for a^*b ($q_1 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_2$). Making q_1 accepting accepts $a \notin (a^*b)^*$. The new start state has no incoming edges, so a run cannot stop there half-way.
- **Concatenation with $\emptyset$** (2023-24 Q1(c)): the NFA for $\emptyset$ has no accepting state, so the concatenation NFA has $F = \emptyset$ and accepts nothing: $L \circ \emptyset = \emptyset$.
- **Evaluate** (2020 Q3(a)): $\emptyset^*\emptyset^*\emptyset^*\emptyset^* = \{\varepsilon\}$; $\emptyset^* \cup \{\varepsilon\} = \{\varepsilon\}$; $(\emptyset^* \cup 1)\{0, 1\} = \{0, 1, 10, 11\}$; $(11)^*\emptyset(00)^* = \emptyset$; $(\emptyset^* \cup 0)\{0, 1, 11\} = \{0, 1, 11, 00, 01, 011\}$.
- **Simplify** (2019-20 Q2(b)): $\emptyset^* \cup a^* \cup b^* \cup (a \cup b)^* = (a \cup b)^*$; $((a^*b^*)^*(b^*a^*)^*)^* = (a \cup b)^*$; $(a^*b)^* \cup (b^*a)^* = (a \cup b)^*$ (every non-empty string ends in b or in a).
- **Describe** $(1 + \varepsilon)(00^*1)^*0^*$ (2020 Q4(a)): all binary strings with no two consecutive 1s.
- **Finite or infinite** (2021-22 Q2(c)): $L_1^* \circ L_2^*$ infinite (L_1^* is infinite and $\varepsilon \in L_2^*$); $L_1 \circ (L_1 \cup L_2)$ finite (at most 3×6 strings); $L_2 \circ L_3$ infinite (L_3 is infinite, $L_2 \neq \emptyset$).

5 Pumping lemma (regular)

What they ask: 9 questions. Prove non-regular (2023-24 Q4(c) $a^{n!}$, 2019-20 Q4(c)); criticise an argument (2017-18 Q3(b), 2020 Q1(b), 2021-22 Q1(d), 2019-20 Q1(a)); pumping lengths (2022-23 Q2(b)); why $|xy| \leq p$ (2023-24 Q2(a)); why only negative (2021-22 Q4(a)). All answered in the Proofs PDF, Section 4.

Recipe — the five sentences of every non-regularity proof

1. "Assume, for contradiction, that L is regular. Let p be its pumping length."
2. "Choose $s = \cdots \in L$" — written with p; check $s \in L$ and $|s| \geq p$. Put a block of p equal symbols first.
3. "Let $s = xyz$ be *any* split with $|y| > 0$ and $|xy| \leq p$. Since the first p symbols are …, $y = \ldots^k$ with $1 \leq k \leq p$."
4. "Take $i = \ldots$; then $xy^iz = \ldots \notin L$ because …"
5. "This contradicts the pumping lemma. Hence L is not regular."

For criticism questions check: is s in L? is $|s| \geq p$? did they choose y themselves? did they claim "for all i"? did they try to prove a language *regular* with the lemma? (Each is an error.)

Exam answer — 2019-20 Q4(c) **(15 marks)**

Use the pumping lemma to prove that $L = \{a^{2n}b^{3n}a^n \mid n \geq 0\}$ is not regular.

Assume, for contradiction, that L is regular, and let p be its pumping length.
Choose $s = a^{2p}b^{3p}a^p$. It is in L (take $n = p$) and $|s| = 6p \geq p$.
Let $s = xyz$ be any split with $|y| > 0$ and $|xy| \leq p$. The first $2p \geq p$ symbols of s are a's, so x and y lie in the first block: $y = a^k$ with $1 \leq k \leq p$.
Take $i = 2$: $xy^2z = a^{2p+k}b^{3p}a^p$. If this were in L, the b-block b^{3p} would force $n = p$, so the first block would have to be a^{2p}; but it is a^{2p+k} with $k \geq 1$. Hence $xy^2z \notin L$.
This contradicts the pumping lemma. Therefore L is not regular. ■

Exam answer — 2023-24 Q4(c) **(13 marks)**

Show that $L = \{a^{n!} \mid n \geq 1\}$ is not regular.

Assume L is regular with pumping length p. Let $m = \max(p, 2)$ and choose $s = a^{m!} \in L$; $|s| = m! \geq m \geq p$. Any split with $|y| > 0$, $|xy| \leq p$ has $y = a^k$, $1 \leq k \leq p \leq m$. Take $i = 2$: $|xy^2z| = m! + k$, and $m! < m! + k \leq m! + m < m! + m \cdot m! = (m+1)!$. So $m! + k$ lies strictly between two consecutive factorials and is not a factorial: $xy^2z \notin L$. Contradiction; L is not regular. ■

Trap. Never "let $p = 5$"; never pick x, y, z; never write "$xy^iz \notin L$ for all i". A fixed string like 00011000 is worthless (it does not depend on p). The pumping lemma can never show a language *is* regular. If p might not be prime, do not choose a^p for the primes — choose a prime $q > p$.

6 Context-free grammars: design and ambiguity

What they ask: 15 questions. Design with the design idea (2022-23 Q5, 2021-22 Q2(d), 2020 Q5, 2019-20 Q5, 2017-18 Q5); grammar for an inherently ambiguous language and its ambiguity (2023-24 Q5, 2022-23 Q6, 2020 Q6); is this grammar ambiguous / inherently ambiguous (2021-22 Q6, 2019-20 Q6, 2017-18 Q6); why "context-free" (2023-24 Q3(c), 2021-22 Q1(e)).

Recipe — designing a CFG

1. Split the language into cases joined by **union** ($S \to S_1 \mid S_2$) or pieces joined by **concatenation** ($S \to S_1S_2$).
2. Matched counts: **wrap** from the outside in, $X \to aXb \mid \varepsilon$ (nested matching: outer a with outer b). Free part: $C \to cC \mid \varepsilon$. "At least one extra": $A \to aA \mid a$.
3. Write the idea in words before the rules; then derive one string from each case.
4. **Ambiguity:** one string, two leftmost derivations (or two parse trees). That proves the grammar ambiguous.

Ready answers: balanced parentheses $S \to (S)S \mid \varepsilon$; equal 0s and 1s $S \to 0S1S \mid 1S0S \mid \varepsilon$; at least two 1s $S \to A1A1A$, $A \to 0A \mid 1A \mid \varepsilon$; at least one 0 and one 1 $S \to A0A1A \mid A1A0A$, $A \to 0A \mid 1A \mid \varepsilon$ (all checked by program).

Exam answer — 2021-22 Q2(d) **(15 marks)**

Design a context-free grammar for $\{a^i b^j c^k \mid i \neq j+k,\ i,j,k \geq 0\}$. *Merely putting forth the production rules shall not suffice. You need to explain your design idea first.*

Design idea. If $i = j + k$, every c can be matched with an a from the outside in $(a \ldots c)$, and then every b with an a $(a \ldots b)$: $a^k\,(a^j b^j)\,c^k$. So $i \neq j + k$ means the matching leaves something over. Three cases:

1. $i > j + k$: after matching, at least one a is left in the middle.
2. $k \leq i < j + k$: all c's are matched by a's, the remaining a's are matched by b's, and at least one b is left.
3. $i < k$: all a's are matched by c's, and the middle is b^j followed by at least one unmatched c.

Grammar.

$$S \to X \mid Y \mid W$$

$$X \to aXc \mid P, \quad P \to aPb \mid A, \quad A \to aA \mid a \quad \text{(case 1: } a^k\,a^t\,a^e\,b^t\,c^k,\ e \geq 1\text{)}$$

$$Y \to aYc \mid Q, \quad Q \to aQb \mid B, \quad B \to bB \mid b \quad \text{(case 2: } a^k\,a^t\,b^t\,b^e\,c^k,\ e \geq 1\text{)}$$

$$W \to aWc \mid U, \quad U \to bU \mid C, \quad C \to cC \mid c \quad \text{(case 3: } a^i\,b^j\,c^e\,c^i,\ e \geq 1\text{)}$$

Why it is correct. Each case produces $a^*b^*c^*$ with the stated inequality ($i = j + k + e$, $j = t + e$ with $i = k + t$, or $k = i + e$), and every string with $i \neq j + k$ falls in case 1 ($i > j + k$), case 2 ($i < j + k$, $i \geq k$) or case 3 ($i < k$).

Derivations. $aaab$ $(i = 3 > 1)$: $S \Rightarrow X \Rightarrow P \Rightarrow aPb \Rightarrow aAb \Rightarrow aaAb \Rightarrow aaab$. $abcc$ $(i = 1 < 3,\ i < k)$: $S \Rightarrow W \Rightarrow aWc \Rightarrow aUc \Rightarrow abUc \Rightarrow abCc \Rightarrow abcc$. (Checked by program against the definition on all strings up to length 9.)

Exam answer — 2023-24 Q5 (also 2021-22 Q6, 2019-20 Q6) **(15 marks)**

Design a CFG for $\{a^i b^j c^k \mid i = j \text{ or } i = k;\ i,j,k \geq 1\}$ *and show that it is ambiguous.*

Design idea. Union of two cases. $i = j$: match a with b ($A \to aAb \mid ab$), then any c^m, $m \geq 1$. $i = k$: match the outer a with the outer c ($C \to aCc \mid aDc$), with b^j, $j \geq 1$, in the middle.

$$S \to AB \mid C, \qquad A \to aAb \mid ab, \quad B \to cB \mid c, \qquad C \to aCc \mid aDc, \quad D \to bD \mid b.$$

Ambiguity. abc has $i = j = k = 1$, so it is in both cases. Two leftmost derivations:

$$S \Rightarrow AB \Rightarrow abB \Rightarrow abc, \qquad S \Rightarrow C \Rightarrow aDc \Rightarrow abc.$$

They start with different rules, so they give two different parse trees (root children $A\,B$ versus C). Hence the grammar is ambiguous. In fact every grammar for this language is ambiguous (it is *inherently ambiguous*): the strings $a^n b^n c^n$ must be generated by both mechanisms. (Same answer for $\{a^n b^n c^m d^m\} \cup \{a^n b^m c^m d^n\}$: $S \to AB \mid C$, $A \to aAb \mid ab$, $B \to cBd \mid cd$, $C \to aCd \mid aDd$, $D \to bDc \mid bc$; $abcd$ has two derivations.)

Trap. A leftmost and a rightmost derivation of the same tree do *not* show ambiguity — you need two different trees. "Ambiguous" is about a grammar; "inherently ambiguous" is about a language (every grammar for it is ambiguous). For $E \to E + E \mid E \times E \mid (E) \mid a$ (2017-18 Q6) use $a + a \times a$ and fix it with $E \to E + T \mid T$, $T \to T \times F \mid F$, $F \to (E) \mid a$. **Why "context-free":** a variable is rewritten by $A \to \gamma$ regardless of the symbols around it (its context); a context-sensitive rule $\alpha A \beta \to \alpha \gamma \beta$ may rewrite A only between α and β.

Part B — Atif Sir

7 CNF and the CYK algorithm

What they ask: 7 questions. CYK with the table (2023-24 Q6, 2022-23 Q7, 2020 Q7, 2019-20 Q7, 2017-18 Q7(b)) — almost every paper; convert to CNF (2021-22 Q5, 2017-18 Q7(a)).

Recipe — CYK

Let $w = a_1 \cdots a_n$; X_{ij} = set of variables that derive $a_i \cdots a_j$.

1. Bottom row: $X_{ii} = \{A \mid A \to a_i\}$.
2. Row of length ℓ: $X_{ij} = \{A \mid A \to BC,\ B \in X_{ik},\ C \in X_{k+1,j} \text{ for some } i \le k < j\}$ — try *every* split k.
3. $w \in L(G) \iff S \in X_{1n}$. Draw the triangle (bottom row = single symbols), show at least the computation for the top cell.

CNF order: remove ε-rules (nullable variables) → unit rules → useless symbols → terminals in long bodies ($U_a \to a$) → break bodies longer than 2.

Exam answer — 2022-23 Q7 **(15 marks)**

For the CNF grammar $S \to AB \mid AY \mid YA \mid 1$, $A \to ZA \mid YA \mid 0 \mid 1$, $B \to YA$, $Z \to 0$, $Y \to 1$, use the CYK algorithm to determine whether $0100 \in L(G)$. *Show the table.*

Row 1 (single symbols). 0: $A \to 0$, $Z \to 0 \Rightarrow \{A, Z\}$. 1: $S \to 1$, $A \to 1$, $Y \to 1 \Rightarrow \{A, S, Y\}$.
Row 2. X_{12} ("01"): $\{A, Z\} \times \{A, S, Y\}$ contains ZA ($A \to ZA$) and AY ($S \to AY$) $\Rightarrow \{A, S\}$. X_{23} ("10"): $\{A, S, Y\} \times \{A, Z\}$ contains YA ($A, B, S \to YA$) $\Rightarrow \{A, B, S\}$. X_{34} ("00"): $\{A, Z\} \times \{A, Z\}$ contains ZA $\Rightarrow \{A\}$.
Row 3. X_{13}: splits $X_{11}X_{23}$ gives $ZA \to A$, $AB \to S$; so $\{A, S\}$. X_{24}: split $X_{22}X_{34}$ gives $YA \to A, B, S$; so $\{A, B, S\}$.
Row 4. X_{14}: split $k = 1$: $X_{11} \times X_{24} = \{A, Z\} \times \{A, B, S\}$ gives AB (S) and ZA (A) $\Rightarrow \{A, S\}$.

$\ell = 4$	$\{A, S\}$			
$\ell = 3$	$\{A, S\}$	$\{A, B, S\}$		
$\ell = 2$	$\{A, S\}$	$\{A, B, S\}$	$\{A\}$	
$\ell = 1$	$\{A, Z\}$	$\{A, S, Y\}$	$\{A, Z\}$	$\{A, Z\}$
w	0	1	0	0

Conclusion. $S \in X_{14}$, so $0100 \in L(G)$ (e.g. $S \Rightarrow AB$ with $A \Rightarrow 0$ and $B \Rightarrow 100$). (Here A derives every non-empty string and $L(G)$ = all strings containing at least one 1 — checked by program.)

Exam answer — 2021-22 Q5 **(15 marks)**

Convert $S \to A1A \mid B$, $A \to 0A \mid 1A \mid \varepsilon$, $B \to 1$ to Chomsky normal form, showing the steps.

Step 1 — ε-rules. Nullable: A. Add every version with A deleted, drop $A \to \varepsilon$: $S \to A1A \mid 1A \mid A1 \mid 1 \mid B$, $A \to 0A \mid 1A \mid 0 \mid 1$, $B \to 1$.
Step 2 — unit rules. $S \to B$ is replaced by B's body: $S \to 1$ (already there).
Step 3 — useless symbols. B is no longer reachable from S: drop it. $S \to A1A \mid 1A \mid A1 \mid 1$, $A \to 0A \mid 1A \mid 0 \mid 1$.
Step 4 — terminals in long bodies. $U \to 0$, $V \to 1$: $S \to AVA \mid VA \mid AV \mid 1$, $A \to UA \mid VA \mid 0 \mid 1$.
Step 5 — long bodies. $S \to AVA$ becomes $S \to AD$, $D \to VA$.
CNF. $S \to AD \mid VA \mid AV \mid 1$, $D \to VA$, $A \to UA \mid VA \mid 0 \mid 1$, $U \to 0$, $V \to 1$. (The language is all strings with at least one 1; the CNF grammar and the original agree on all strings up to length 7.)

Trap. In CYK try every split, not only the first one. A cell can be empty — write $\emptyset$. Remove ε-rules *before* unit rules, and generating symbols *before* reachable ones. Keep $S \to \varepsilon$ only if $\varepsilon \in L$.

8 Pushdown automata and the CFL pumping lemma

What they ask: 10 questions. Design a PDA with its transition diagram in *every* paper: $\{a^n b^m c^m d^n\}$ (2023-24 Q8), $\{a^i b^j c^k \mid i = j \text{ or } i = k\}$ (2022-23 Q8, 2020 Q8), ARITH (2021-22 Q7, 2019-20 Q8), $\{a^n b^n c^m d^m\}$ (2017-18 Q8). Also: final state = empty stack (2019-20 Q10(a)); give a non-CFL and prove it (2023-24 Q7, 2021-22 Q8); state and prove the CFL pumping lemma (2022-23 Q9) — proofs in the Proofs PDF, 7–8.

Recipe — designing a PDA (Atif's notation: $a, X/\gamma$ = read a, pop X, push γ, leftmost on top)

1. **Stack strategy** in one sentence: what one stack symbol stands for.
2. Start with Z_0 on the stack; seeing Z_0 on top means "everything matched".
3. One state per **phase** of the input (a-part, b-part, ...). Separate arcs for the *first* symbol of a block when the block must be non-empty.
4. "Or" in the language ⇒ an ε-guess at the start. Missing transition = that branch dies.
5. Say how it accepts (final state or empty stack), draw the diagram, give the 7-tuple, trace one accepted and one rejected string as IDs.

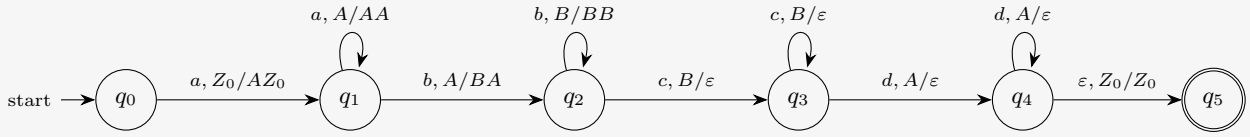
CFG → PDA (empty stack): one state q; $\delta(q, \varepsilon, A) \ni (q, \beta)$ for each rule $A \to \beta$; $\delta(q, a, a) = \{(q, \varepsilon)\}$; start stack S.

Exam answer — 2023-24 Q8 **(15 marks)**

Design a PDA for $\{a^n b^m c^m d^n \mid n \geq 1,\ m \geq 1\}$. Draw its transition diagram.

Idea. Push an A for every a and a B for every b; pop a B for every c, then an A for every d. The B's sit above the A's, so the c's are matched with the b's and the d's with the a's (nested). Reading c with A on top (too many c's) or d with B on top (too few c's) has no move, so that run dies. Accept by final state when Z_0 is back on top.

$$P = (\{q_0, \ldots, q_5\}, \{a, b, c, d\}, \{A, B, Z_0\}, \delta, q_0, Z_0, \{q_5\})$$



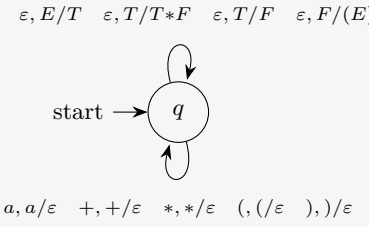
Accepted: *aabcdd*. $(q_0, aabcdd, Z_0) \vdash (q_1, abcdd, AZ_0) \vdash (q_1, bcdd, AAZ_0) \vdash (q_2, cdd, BAAZ_0) \vdash (q_3, dd, AAZ_0) \vdash (q_4, d, AZ_0) \vdash (q_4, \varepsilon, Z_0) \vdash (q_5, \varepsilon, Z_0)$ — accept.
Rejected: *aabccd*. $\cdots \vdash (q_2, ccd, BAAZ_0) \vdash (q_3, cd, AAZ_0)$: reading c with A on top has no move — the only run dies, reject. $n, m \geq 1$ is enforced by the separate first-a and first-b arcs. (Checked by program on all strings up to length 9.)

Exam answer — 2021-22 Q7 and 2019-20 Q8 **(15 marks)**

Construct a PDA for ARITH, the legal arithmetic expressions over $\{a, +, *, (,)\}$ (valid: $a + a * a$, $((a + a) * (a + a)) + (a)$; invalid: $aa * a$, $a + +a$, $+a$, $(a + a))$). Give its transition diagram.

Idea. ARITH is generated by the (unambiguous) grammar $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to (E) \mid a$. Convert it to a PDA that simulates leftmost derivations: expand the variable on top of the stack, match terminals with the input. Accept by empty stack.

$$P = (\{q\}, \{a, +, *, (,)\}, \{E, T, F, a, +, *, (,)\}, \delta, q, E)$$



Accepted: $a + a * a$. $(q, a+a*a, E) \vdash (q, a+a*a, E+T) \vdash (q, a+a*a, T+T) \vdash (q, a+a*a, F+T) \vdash (q, a+a*a, a+T) \vdash (q, +a*a, +T) \vdash (q, a*a, T) \vdash (q, a*a, T*F) \vdash (q, a*a, F*F) \vdash (q, a*a, a*F) \vdash$

$(q, *a, *F) \vdash (q, a, F) \vdash (q, a, a) \vdash (q, \varepsilon, \varepsilon)$ — accept.
Rejected: $a{+}{+}a$. After matching $a+$ the stack holds T (or more); every expansion of T puts a or (on top, never +, so the second + can never be matched: every run dies. (For acceptance by final state, add the empty-stack → final-state construction.)

Exam answer — 2022-23 Q8 and 2020 Q8 **(15 marks)**

Design a PDA for $\{a^i b^j c^k \mid i = j \text{ or } i = k\}$.

Idea. Guess at the start which equality holds. Branch $i = j$: push A per a, pop per b, back at Z allow any c's. Branch $i = k$: push A per a, skip the b's, pop per c. Accept by final state (p_2, r_3).

start: $\delta(q_0, \varepsilon, Z) = \{(p_0, Z), (r_0, Z)\}$
$i = j$: $(p_0, a, Z/AZ)$, $(p_0, a, A/AA)$, $(p_0 \to p_1, b, A/\varepsilon)$, $(p_1, b, A/\varepsilon)$, $(p_0 \to p_2, \varepsilon, Z/Z)$, $(p_1 \to p_2, \varepsilon, Z/Z)$, $(p_2, c, Z/Z)$
$i = k$: $(r_0, a, Z/AZ)$, $(r_0, a, A/AA)$, $(r_0 \to r_1, \varepsilon, X/X)$, $(r_1, b, X/X)$, $(r_1 \to r_2, c, A/\varepsilon)$, $(r_2, c, A/\varepsilon)$, $(r_1 \to r_3, \varepsilon, Z/Z)$, $(r_2 \to r_3, \varepsilon, Z/Z)$ $(X \in \{A, Z\})$

Trace $aabcc$ **(branch** $i = k$**).** $(q_0, aabcc, Z) \vdash (r_0, aabcc, Z) \vdash (r_0, abcc, AZ) \vdash (r_0, bcc, AAZ) \vdash (r_1, bcc, AAZ) \vdash (r_1, cc, AAZ) \vdash (r_2, c, AZ) \vdash (r_2, \varepsilon, Z) \vdash (r_3, \varepsilon, Z)$ — accept. aab: branch 1 is left with A on top, branch 2 has no c — reject. (2017-18 Q8, $\{a^n b^n c^m d^m\}$: push per a, pop per b, then push per c, pop per d, with first-symbol arcs for $n, m \geq 1$.)

Exam answer — 2023-24 Q7 and 2021-22 Q8 **(15 marks)**

Give an example of a language that is not context-free, and prove it using the pumping lemma for CFLs.

Example. $L = \{0^n 1^n 2^n \mid n \geq 1\}$.
Lemma (HMU). If L is a CFL there is n such that every $z \in L$, $|z| \geq n$, is $z = uvwxy$ with $|vwx| \leq n$, $vx \neq \varepsilon$ and $uv^i wx^i y \in L$ for all $i \geq 0$.
Proof. Assume L is a CFL with constant n. Choose $z = 0^n 1^n 2^n \in L$, $|z| = 3n \geq n$. Let $z = uvwxy$ be any split with $|vwx| \leq n$, $vx \neq \varepsilon$. Since $|vwx| \leq n$, vwx cannot contain both a 0 and a 2 (all n 1s lie between them). (a) If v or x contains two different symbols, uv^2wx^2y has them out of order, so it is not in $0^*1^*2^*$. (b) Otherwise, if vwx has no 2, then uwy ($i = 0$) still has n 2s but fewer 0s or fewer 1s; if vwx has no 0, uwy has n 0s but fewer 1s or 2s. In every case some $uv^i wx^i y \notin L$, contradicting the lemma. Hence L is not context-free. ■

Trap. Say what each stack symbol means. Show the IDs $(q, \text{remaining input}, \text{stack})$ — the trace earns marks. A PDA needs nondeterminism for "or" languages and for ww^R; say so. In the CFL lemma you choose z and i only; pump v and x *together*; cover every placement of vwx.

9 Turing machine design on $w_1 c w_2 c$

What they ask: 6 questions, one in every paper, all on the same pattern: AND (2019-20 Q9), OR (2017-18 Q9), XOR (2020 Q9), equality test (2023-24 Q9), NOT after c (2022-23 Q10), signed magnitude → 2's complement (2021-22 Q9). Multiple tracks and storage in the state are allowed.

Recipe — a bit-by-bit TM

1. **Algorithm in words:** take the leftmost unmarked bit of w_1, remember it in the state, mark it; walk to the leftmost unmarked bit of w_2, combine, mark it; walk past the second c to the first blank and write the result bit; return to the left end; repeat.
2. **Marking:** $0 \mapsto X$, $1 \mapsto Y$ (or a check mark on a second track). **State storage:** p_b, t_o carry a bit.
3. **Stop:** when the leftmost-unmarked search reaches the first c; restore $X \to 0$, $Y \to 1$ if the output must show w_1, w_2.
4. Give the 7-tuple, the transition table (only the moves that occur), and a sweep-by-sweep trace of a short input.

Exam answer — 2019-20 Q9 **(15 marks)**

Design a TM computing the AND of equal-length binary w_1 and w_2. Initially the tape is w_1cw_2c; finish with their AND in binary after the second c.

Idea. Repeat: mark the leftmost unmarked bit b of w_1 ($0 \to X$, $1 \to Y$) and remember it in the state (p_b); go past the first c to the leftmost unmarked bit b' of w_2, mark it and remember $o = b \wedge b'$ (t_o); go past the second c and the result bits to the first blank and write o; go back to the left end. When the search in w_1 meets c, every bit is done: restore the marks and halt.

$$M = (\{q_0, p_0, p_1, s_0, s_1, t_0, t_1, u_0, u_1, k, \ell, r, h\}, \{0, 1, c\}, \{0, 1, c, X, Y, B\}, \delta, q_0, B, \{h\})$$

state (meaning)	moves $\delta(\text{state}, \text{read}) = (\text{state}, \text{write}, \text{dir})$
q_0 (find next bit of w_1)	$X, Y \to$ same, R; $0 \to (p_0, X, R)$; $1 \to (p_1, Y, R)$; $c \to (\ell, c, L)$
p_b (carry b to w_2)	$0, 1 \to$ same, R; $c \to (s_b, c, R)$
s_b (find next bit of w_2)	$X, Y \to$ same, R; $0 \to (t_{b\wedge 0}, X, R)$; $1 \to (t_{b\wedge 1}, Y, R)$
t_o (go past 2nd c)	$0, 1 \to$ same, R; $c \to (u_o, c, R)$
u_o (write result)	$0, 1 \to$ same, R; $B \to (k, o, L)$
k (back to left end)	$0, 1, c, X, Y \to$ same, L; $B \to (q_0, B, R)$
ℓ (done: to left end)	$X, Y, c \to$ same, L; $B \to (r, B, R)$
r (restore)	$X \to (r, 0, R)$; $Y \to (r, 1, R)$; $0, 1, c \to$ same, R; $B \to (h, B, R)$

Trace on $101c110c$ (tape after each round): $101c110c \vdash^* Y01cY10c\mathbf{1} \vdash^* YX1cYY0c\mathbf{10} \vdash^* YXYcYYXc10\mathbf{0} \vdash^* 101c110c100$ (restore, halt). Round 1: $1 \wedge 1 = 1$; round 2: $0 \wedge 1 = 0$; round 3: $1 \wedge 0 = 0$. Result $100 = 101 \wedge 110$. (Checked by program on all equal-length pairs of length 1–5.)

The same machine for the other papers — change only these lines

- **OR (2017-18 Q9,** $N_1\#N_2\#$**) / XOR (2020 Q9):** in s_b go to $t_{b\vee b'}$ / $t_{b\oplus b'}$.
- **Equality (2023-24 Q9):** in s_b, if $b' = b$ mark and go back (k); if $b' \neq b$ go right past the second c and write 1, halt. If q_0 reaches the first c (all bits matched), go past the second c and write 0, halt.
- **NOT after** c **(2022-23 Q10, tape** wc**):** no second word: from p_b go past c to the first blank, write $1 - b$, come back.
- **Signed magnitude → 2's complement (2021-22 Q9, MSB 1):** go to the right end; moving left, copy bits up to and including the first 1; after that invert every remaining magnitude bit; leave the sign bit 1. (Check: 1 0110 (-6) → 1 1010.)

10 Turing-machine variants and simulations

What they ask: 6 questions, every paper: how a one-tape TM with tracks and state storage simulates a multitape TM (2023-24 Q10, 2019-20 Q10(b)); true/false on simulations with polynomial cost (2022-23 Q11, 2021-22 Q10, 2020 Q10, 2017-18 Q10).

Exam answer — 2023-24 Q10 **(15 marks)**

What is a multitape TM? Explain how a one-tape TM with multiple tracks and storage in the state can simulate it.

Multitape TM. A finite control with k tapes, each with its own head. Input on tape 1, other tapes blank. One move depends on the state and all k scanned symbols; it changes state, writes on each tape and moves each head independently (L, R or S).

Simulation. One-tape N uses $2k$ tracks: for each tape of M, one track holds its contents and one track holds a single marker X under that tape's head. Example for $k = 2$:

head 1			X		
tape 1	A_1	A_2	A_3	A_4	$\cdots$
head 2		X			
tape 2	B_1	B_2	B_3	$\cdots$	

N's state stores M's state and a count of how many head markers lie to its left. To simulate one move of M:

(1) N sweeps right from the leftmost marker, visiting all k markers and storing the k scanned symbols in its state; now it knows M's move. (2) It sweeps back, at each marker writes the new symbol on the contents track and moves the marker one cell left/right as M would. (3) It records M's new state. N accepts when the stored state of M is accepting. So $L(N) = L(M)$.
Cost. After n moves the markers are at most $2n$ cells apart, so one simulated move takes $O(n)$ moves and n moves take $O(n^2)$.

True / false answers (justify in two lines each)

- One-tape TM with multiple tracks (and state storage) simulates a multitape TM — **True**, $2k$ tracks, $O(n^2)$ time.
- A DTM simulates n steps of a computer in $p(n)$ steps — **True** (if each instruction lengthens a word by at most 1 bit): $O(n^3)$ multitape, $O(n^6)$ one-tape. Memory is (address, value) pairs on tape 1, at most $O(n^2)$ cells; each instruction costs $O(n^2)$.
- A DTM (or a computer) simulates n moves of an NTM in $p(n)$ moves — **Not known; believed false.** The known method is breadth-first search of the computation tree: up to $(n+1)m^n$ IDs, exponential. A polynomial simulation would give P = NP.

11 Undecidability

What they ask: 7 questions, every paper: L_d is not RE (2022-23 Q12, 2017-18 Q11); the halting problem is RE but not recursive (2019-20 Q12, 2023-24 Q11, 2021-22 Q11); definitions and one example per class (2020 Q11, 2019-20 Q11). Notation as in Atif Sir's slides (Hopcroft).

Recipe — the three proofs to know

L_d **not RE** (diagonalisation): if $L_d = L(M_i)$, ask whether $w_i \in L_d$ — both answers contradict the definition. L_u **not recursive:** a decider for L_u gives one for $\overline{L_u}$, which gives a TM for L_d (input $w \mapsto w111w$). **Halting not recursive:** a decider for H would decide L_u ("does M halt? if yes, run it"). **RE:** simulate (universal TM).
Reduction rule: if P_1 reduces to P_2 and P_1 is not recursive (not RE), then P_2 is not recursive (not RE).

Exam answer — 2019-20 Q12 (also 2023-24 Q11, 2021-22 Q11) (15 marks)

Define the halting problem. Prove that it is recursively enumerable but not recursive.

Definition. For a TM M, $H(M)$ is the set of inputs on which M halts (accepting or not). The halting problem is $H = \{(M, w) \mid w \in H(M)\}$.
H is RE. Build U_H like the universal TM: on input (M, w) simulate M on w; if M halts (in any state), accept. If M runs forever, U_H runs forever, which is allowed. So $L(U_H) = H$ and H is RE.
H is not recursive. We use that $L_u = \{(M, w) \mid M \text{ accepts } w\}$ is not recursive. Suppose, for contradiction, that H is recursive, decided by a TM D that always halts. Then decide L_u as follows. On input (M, w): (1) run D on (M, w); if D says M does not halt on w, reject (a machine that never halts never accepts); (2) otherwise simulate M on w — it is guaranteed to halt — and accept iff it halts in an accepting state. This procedure always halts and accepts exactly L_u, so L_u would be recursive — a contradiction. Hence H is not recursive. (We have reduced L_u to H.) ■

Exam answer — 2022-23 Q12 (also 2017-18 Q11) (15 marks)

Define the diagonalization language L_d and prove that it is not recursively enumerable.

Definition. Order binary strings $w_1 = \varepsilon, w_2 = 0, w_3 = 1, w_4 = 00, \ldots$ ($1w_i$ is i in binary). Every TM with input alphabet $\{0, 1\}$ has a binary code ($\delta(q_i, X_j) = (q_k, X_l, D_m)$ coded $0^i10^j10^k10^l10^m$, transitions separated by 11); M_i is the TM with code w_i (the TM with no moves if w_i is not a valid code). $L_d = \{w_i \mid w_i \notin L(M_i)\}$ — the complement of the diagonal of the table "M_i accepts w_j".
Proof. Suppose $L_d = L(M)$ for some TM M. Its input alphabet is $\{0, 1\}$, so $M = M_i$ for some i. If $w_i \in L_d$ then M_i accepts w_i, so by definition $w_i \notin L_d$. If $w_i \notin L_d$ then M_i does not accept w_i, so by definition $w_i \in L_d$. Both cases are contradictions, so no TM accepts L_d: L_d is not RE. ■

One example per class (2019-20 Q11, 2020 Q11)

Regular: 0^*1^*. CF not regular: $\{0^n1^n\}$. Recursive not CF: $\{0^n1^n2^n\}$ (a marking TM always halts; not CF by the pumping lemma). RE not recursive: L_u or H. Not RE: L_d (or $\overline{L_u}$).
RE: $L = L(M)$ for some TM. Recursive: $L = L(M)$ for a TM that halts on every input. If L and $\overline{L}$ are both RE, L is recursive.

12 Complexity: Cook's theorem, NP-completeness, space

What they ask: 17 questions. Cook's theorem and its clauses in 4 papers (2023-24 Q13, 2022-23 Q13(i), 2021-22 Q12, 2020 Q12(a)); class relationships, Venn diagram, PSPACE vs NPSPACE in every paper (2023-24 Q12, 2022-23 Q13(ii), 2021-22 Q13, 2020 Q12(b), 2019-20 Q13, 2017-18 Q13); NP-completeness proofs (2017-18 Q1(b), Q12) and an NP certificate (2020 Q1(a)).

Exam answer — 2023-24 Q13 **(15 marks)**

Explain how Cook's theorem shows that every problem in NP reduces to SAT. An NTM has $\delta(q,t) = \{(q_1,t_1,R),(q_2,t_2,L)\}$; construct its Boolean-formula rule under Cook's construction when the head is on cell i at step j (not at either end).

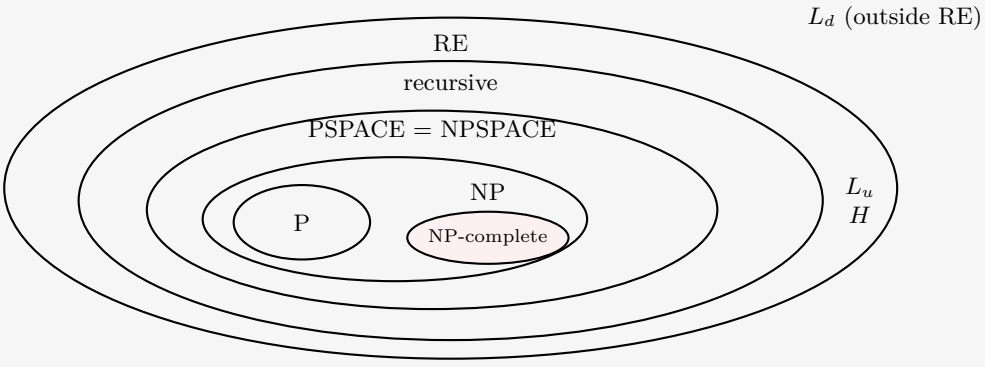
Cook's theorem: SAT is NP-complete. SAT $\in$ NP: guess an assignment, evaluate in polynomial time. For any $L \in$ NP take an NTM M with at most $p(n)$ moves on every branch. For input w build $F(M,w)$, satisfiable iff some branch accepts; an assignment describes one branch. Variables: $s(q,j)$ (state q at step j), l_{ij} (head on cell i at step j), $c(i,j,t)$ (cell i holds t at step j), $u(i,j)$ (cell i unchanged from j to $j+1$), for $0 \le i,j \le p(n)$. Seven rule families: (1) exactly one state per step; (2) exactly one head position; (3) exactly one symbol per cell; (4) unchanged cells keep their symbol; (5) each step follows δ; (6) step 0 is the initial ID $q_0\,w\,B^{p(n)+1-n}$; (7) the state at step $p(n)$ is accepting. $F(M,w)$ is their AND; the largest family (5) has size $O(|Q||\Gamma|p(n)^3)$, so F is built in polynomial time and $w \in L \iff F(M,w)$ is satisfiable. So every NP language reduces to SAT.
The rule. Here $d_1 = R$, $d_2 = L$, so the new head positions are $i+1$ and $i-1$:

$$\big[s(q,j) \wedge l_{ij} \wedge c(i,j,t)\big] \to \bigwedge_{k \neq i} u(k,j) \;\wedge\; \Big[\big(s(q_1,j{+}1) \wedge l_{(i+1)(j+1)} \wedge c(i,j{+}1,t_1)\big) \vee \big(s(q_2,j{+}1) \wedge l_{(i-1)(j+1)} \wedge c(i,j{+}1,t_2)\big)\Big].$$

In words: if at step j the state is q and the head reads t on cell i, then every other cell is unchanged and at step $j+1$ either (state q_1, cell i holds t_1, head on $i+1$) or (state q_2, cell i holds t_2, head on $i-1$). The $\vee$ is where the satisfying assignment chooses the branch.

Exam answer — 2023-24 Q12, 2022-23 Q13(ii), 2020 Q12(b), 2019-20 Q13 **(7–15 marks)**

Draw a Venn diagram of P, NP, NP-complete, PSPACE, recursive and RE. What is the relationship between PSPACE and NPSPACE? Justify.



Justification. P $\subseteq$ NP: a DTM is an NTM with no choices. NP $\subseteq$ PSPACE: in $T(n)$ moves at most $T(n)+1$ cells are visited, so NP $\subseteq$ NPSPACE $=$ PSPACE. NP-complete problems are in NP and, if

P $\neq$ NP, outside P. Every class here is decided by machines that always halt (a $p(n)$-space machine can be stopped after $c^{1+p(n)}$ moves), so it lies inside the recursive languages; L_u, H are RE but not recursive; L_d is not RE. Proper inclusions P $\neq$ NP $\neq$ PSPACE are believed, not proved.

PSPACE = NPSPACE (Savitch). PSPACE $\subseteq$ NPSPACE trivially. Conversely an $f(n)$-space NTM is simulated deterministically by REACH(I, J, m): "can ID I reach ID J in $\leq m$ moves?", answered by trying every middle ID K and recursing on $(I, K, m/2)$ and $(K, J, m/2)$ one after the other. With $m = c^{1+f(n)}$ the recursion depth is $\log_2 m = O(f(n))$ and each frame is $O(f(n))$ cells, so the space is $O(f^2(n))$ — polynomial when f is. Hence PSPACE = NPSPACE.

Exam answer — 2017-18 Q12 **(15 marks)**

Prove that the subgraph-isomorphism problem (does G_1 contain a copy of G_2 as a subgraph?) is NP-complete.

In NP. Certificate: a one-to-one map $f : V(G_2) \to V(G_1)$. Check in polynomial time that f is one-to-one and that $(f(u), f(v)) \in E(G_1)$ for every edge $(u, v) \in E(G_2)$.

NP-hard: CLIQUE $\leq_p$ SUBISO. CLIQUE is NP-complete. Map $\langle G, k\rangle$ to $\langle G_1, G_2\rangle = \langle G, K_k\rangle$ (K_k = complete graph on k vertices, written in $O(k^2)$ time). If G has a k-clique S, any bijection $V(K_k) \to S$ maps every edge of K_k onto an edge of G, so G contains K_k. If K_k embeds in G by f, the k image vertices are pairwise adjacent, a k-clique. So $\langle G, k\rangle \in$ CLIQUE $\iff$ $\langle G, K_k\rangle \in$ SUBISO.

Conclusion. SUBISO is in NP and an NP-complete problem reduces to it in polynomial time; hence it is NP-complete. ■

Trap. Reduce *from* the known NP-complete problem *to* the new one. Prove both directions and say the reduction is polynomial. A certificate answer (Dominating Set, 2020 Q1(a)) has three parts: what the certificate is, that it is polynomial size, and the polynomial-time check (every vertex is in D or next to D, $|D| \leq k$). "SAT in P $\Rightarrow$ P = NP" is **true**; "NP $\supseteq$ PSPACE" is **not known and believed false**.

Last evening before the exam. (1) Redo the exam answers in Topics 2, 3, 6 (Hakim) and 7, 8, 9 (Atif) on paper without looking — these are the design questions that appear every year. (2) Learn the five-sentence pumping proof and the three undecidability proofs by heart. (3) Read Cook's seven rules and the Venn diagram once more. (4) Skim the short theory answers in Topics 2 and 4.